DSA2020: ARTIFICIAL INTELLIGENCE

PROJECT: TIMETABLE GENERATOR

BY: Harina Chohan (666983)

Assume that you are in charge of creating a laboratory schedule for the 5 Laboratories in the LKB Building at UISU-A. Assume that we offer 200 classes in a semester and have 60 faculty members. Demonstrate how you have factored in constraints such as:

(i) ensuring classes taken in the same semester do not take place at the same time,

(ii) classes taking place at the same time are taught by different faculty,

(ii) concentration classes are scheduled in smaller-sized labs,

(iii) Double classes take place on Mon/Wed or Tue/Thur etc.

```
# Librarries
import tkinter as tk # GUI library
from tkinter import ttk, messagebox, scrolledtext
import pandas as pd
import numpy as np
import random
from datetime import datetime, timedelta
import json

class LaboratoryScheduler:
    def __init__(self, root):
        self.root = root
        self.root.title("UISU-A LKB Building Laboratory Scheduler")
        self.root.geometry("1000x800")  # 1size of window

        # Initializing the labs and their capacities
        # The smallest 2 labs: Lab 3 and 4 are the labs for
concentration classes
        self.labs = {
            'Lab 1': {'capacity': 40, 'type': 'general'},
            'Lab 2': {'capacity': 35, 'type': 'general'},
            'Lab 3': {'capacity': 20, 'type': 'concentration'},
            'Lab 4': {'capacity': 25, 'type': 'concentration'},
            'Lab 5': {'capacity': 30, 'type': 'general'}
        }

        self.time_slots = self.generate_time_slots()
        self.classes = self.generate_sample_classes()
        self.faculty = self.generate_sample_faculty()
        self.schedule = pd.DataFrame()
```

```
        self.setup_gui()

    # Function for the time slots
    # In USIU-A classes start at 7am and end at 9pm
    def generate_time_slots(self):
        # Normal school week is Monday to Thursday
        """Generate time slots for Mon-Thurs, 7AM-9PM"""
        days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',
'Friday']
        slots = []
        for day in days:
            for hour in range(7, 21):  # 7AM to 9PM
                # Add to the class slot list the day and timing of
class
                # Classes happen every 2 hours (hence +2)
                slots.append(f"{day} {hour:02d}:00-{hour+2:02d}:00")
        return slots

    def generate_sample_classes(self):
        """Sample classes"""
        classes = []
        # Sample of some courses that can be offered in USIU-A that
would be held in LKB
        subjects = ["DSA2020A","DSA2020B" "APT1050A", "APT1050B",
"APT1050C", "APT1030", "APT1040", "APT3010B", "APT3080", "IST1025",
"IST3020", "IST3050",
                    "DSA3050", "DSA1060A", "DSA1060B", "DSA1080",
"IST1010", "DSA3040", "FIC4020", "IST2010", "DSA3020", "IST4030",
"IST2045", "APT3060","ENT1010", "IRL3005",
                    "MTH1060","MTH1105", "IST1010B", "MTH1110",
"DSA4010", "SWE4060", "IST1020B"
                    ]
        # Since assuming that the number of classes offered in a
semester is 200:
        for i in range(200):
            class_type = random.choice(['concentration', 'double']) #
Classes can be a concentration class once a week or a double class on
MW or TR
            subject = random.choice(subjects) # Randomly select which
class is a double class and which one is a concentration class

            semester = 'Fall'  # Next semester is the fall semester
            # Add to the classes list
            classes.append({
                'id': f'CLASS_{i+1:03d}',  # Class number up to 3
digits (1 to 200)
                'name': f'{subject}',   # Name of course
                'type': class_type,  # Concentration or double class
                # if a concentration class then the minimum capacity
```

```
is 8 students and maximum is 25 (the maximum a concentration lab can
facilitate)
                # if a double class, the minimum number of students is
15 while maximum is 40 (maximum capacity of lab 1)
                'capacity_needed': random.randint(15, 40) if
class_type != 'concentration' else random.randint(8, 25),
                'semester': semester,
                # Concentration classes are once a week
                'duration': 2 if class_type == 'double' else 1,
                # Concentration classes are only on Fridays
                # However there is a 20% chance that a concentration
class is not on a Friday
                'preferred_days': ['Monday', 'Wednesday'] if
class_type == 'double' and random.random() > 0.80
                                else ['Tuesday', 'Thursday'] if
class_type == 'double'
                                else None
            })
        # Data frame of the classes list above
        return pd.DataFrame(classes)

    def generate_sample_faculty(self):
        """Generate sample faculty members"""
        faculty = []
        # Different major departments in LKB
        departments = ['Data Science', 'APT', 'IST', 'Software
Engineering']

        # Not lecturer names but a list of faculty from 1 to 60
        for i in range(60):
            faculty.append({
                'id': f'FAC_{i+1:02d}',
                'name': f'Dr. Faculty {i+1}',   # No names in the
system so 1 to 60 faculty members
                'department': random.choice(departments),  # Randomly
placed into one of the 4 departments listed
                'max_hours_per_day': random.randint(2, 6), # A faculty
member has to at least teach 1 class a day and at most 3 classes
                'availability': random.sample(self.time_slots,
random.randint(20, 40))
            })
        # Data frame of the faculty list above
        return pd.DataFrame(faculty)
    # Main GUI setup
    def setup_gui(self):
        """Setup the main GUI"""
        # Create notebook for tabs
        notebook = ttk.Notebook(self.root) # ttk.Notebook widget
        # Allows the notebook to occupy if space is available
```

```
        # 10-pixel margin around its edges
        notebook.pack(fill='both', expand=True, padx=10, pady=10)

        # Setup tabs
        # The GUI will have a data, constraints, schedule and an
analysis tab
        self.setup_data_tab(notebook)
        self.setup_constraints_tab(notebook)
        self.setup_schedule_tab(notebook)
        self.setup_analysis_tab(notebook)

    def setup_data_tab(self, notebook):
        """Setup data management tab"""
        data_frame = ttk.Frame(notebook)
        notebook.add(data_frame, text="Data Management")

        # Labs section
        labs_frame = ttk.LabelFrame(data_frame, text="Laboratories")
        labs_frame.pack(fill='x', padx=5, pady=5)

        labs_tree = ttk.Treeview(labs_frame, columns=('Capacity',
'Type'), show='tree headings', height=6)
        labs_tree.heading('#0', text='Lab Name')
        labs_tree.heading('Capacity', text='Capacity')
        labs_tree.heading('Type', text='Type')

        for lab_name, info in self.labs.items():
            labs_tree.insert('', 'end', text=lab_name,
values=(info['capacity'], info['type']))

        labs_tree.pack(fill='x', padx=5, pady=5)

        # Classes section
        classes_frame = ttk.LabelFrame(data_frame, text="Classes
Overview")
        classes_frame.pack(fill='both', expand=True, padx=5, pady=5)

        # Summary statistics
        stats_frame = ttk.Frame(classes_frame)
        stats_frame.pack(fill='x', padx=5, pady=5)

        total_classes = len(self.classes)
        concentration_classes = len(self.classes[self.classes['type']
== 'concentration'])
        double_classes = len(self.classes[self.classes['type'] ==
'double'])

        ttk.Label(stats_frame, text=f"Total Classes:
{total_classes}").pack(side='left', padx=10)
```

```
        ttk.Label(stats_frame, text=f"Concentration:
{concentration_classes}").pack(side='left', padx=10)
        ttk.Label(stats_frame, text=f"Double:
{double_classes}").pack(side='left', padx=10)

        # Faculty section
        faculty_frame = ttk.LabelFrame(data_frame, text="Faculty
Overview")
        faculty_frame.pack(fill='x', padx=5, pady=5)

        ttk.Label(faculty_frame, text=f"Total Faculty Members:
{len(self.faculty)}").pack(padx=5, pady=5)

        # Action buttons
        button_frame = ttk.Frame(data_frame)
        button_frame.pack(fill='x', padx=5, pady=5)

        ttk.Button(button_frame, text="Generate New Data",
command=self.regenerate_data).pack(side='left', padx=5)
        ttk.Button(button_frame, text="Export Data",
command=self.export_data).pack(side='left', padx=5)

    def setup_constraints_tab(self, notebook):
        """Setup constraints configuration tab"""
        constraints_frame = ttk.Frame(notebook)
        notebook.add(constraints_frame, text="Constraints")

        # Constraint checkboxes
        self.constraint_vars = {}
        # The different constraints required are check boxes that can
be checked on and off
        constraints = [
            ('no_time_conflict', 'No time conflicts for same semester
classes'),
            ('faculty_availability', 'Respect faculty availability'),
            ('concentration_small_labs', 'Concentration classes in
smaller labs'),
            ('double_class_days', 'Double classes on Mon/Wed or
Tue/Thu'),
            ('lab_capacity', 'Respect laboratory capacity limits'),
            ('faculty_max_hours', 'Respect faculty maximum hours per
day')
        ]

        constraint_frame = ttk.LabelFrame(constraints_frame,
text="Active Constraints")
        constraint_frame.pack(fill='x', padx=10, pady=10)

        for var_name, description in constraints:
```

```
            self.constraint_vars[var_name] = tk.BooleanVar(value=True)
            ttk.Checkbutton(constraint_frame, text=description,

variable=self.constraint_vars[var_name]).pack(anchor='w', padx=5,
pady=2)

        # Constraint parameters
        params_frame = ttk.LabelFrame(constraints_frame,
text="Parameters")
        params_frame.pack(fill='x', padx=10, pady=10)

        # Max attempts for scheduling
        ttk.Label(params_frame, text="Max Scheduling
Attempts:").grid(row=0, column=0, sticky='w', padx=5, pady=2)
        self.max_attempts_var = tk.StringVar(value="1000")
        ttk.Entry(params_frame, textvariable=self.max_attempts_var,
width=10).grid(row=0, column=1, padx=5, pady=2)

        # Priority weights
        ttk.Label(params_frame, text="Scheduling
Priority:").grid(row=1, column=0, sticky='w', padx=5, pady=2)
        self.priority_var = tk.StringVar(value="concentration_first")
        # The scheduling can prioritize concentration classes or
double or random selection
        priority_combo = ttk.Combobox(params_frame,
textvariable=self.priority_var,
                                      values=["concentration_first",
"double_first", "random"], width=15)
        priority_combo.grid(row=1, column=1, padx=5, pady=2)

    def setup_schedule_tab(self, notebook):
        """Setup scheduling tab"""
        schedule_frame = ttk.Frame(notebook)
        notebook.add(schedule_frame, text="Schedule Generation")

        # Control buttons
        control_frame = ttk.Frame(schedule_frame)
        control_frame.pack(fill='x', padx=10, pady=10)

        # Buttons that generate the schedule based on the constraints,
can clear the schedule and export it as a csv/json file
        ttk.Button(control_frame, text="Generate Schedule",
                   command=self.generate_schedule).pack(side='left',
padx=5)
        ttk.Button(control_frame, text="Clear Schedule",
                   command=self.clear_schedule).pack(side='left',
padx=5)
        ttk.Button(control_frame, text="Export Schedule",
                   command=self.export_schedule).pack(side='left',
```

```
padx=5)

        # Progress bar
        # Showing how the generation of the schedule is progressing
        self.progress_var = tk.DoubleVar()
        self.progress_bar = ttk.Progressbar(control_frame,
variable=self.progress_var,
                                            maximum=100, length=200)
        self.progress_bar.pack(side='right', padx=5)

        # Status label
        self.status_var = tk.StringVar(value="Ready to generate
schedule")
        ttk.Label(schedule_frame,
textvariable=self.status_var).pack(pady=5)

        # Schedule display
        schedule_display_frame = ttk.LabelFrame(schedule_frame,
text="Generated Schedule")
        schedule_display_frame.pack(fill='both', expand=True, padx=10,
pady=10)

        # Create treeview for schedule
        self.schedule_tree = ttk.Treeview(schedule_display_frame,
                                          columns=('Class', 'Faculty',
'Lab', 'Time', 'Type'),
                                          show='headings')

        # Configure columns
        # Columns of class, faculty, lab, timing and the type of class
        self.schedule_tree.heading('Class', text='Class')
        self.schedule_tree.heading('Faculty', text='Faculty')
        self.schedule_tree.heading('Lab', text='Laboratory')
        self.schedule_tree.heading('Time', text='Time Slot')
        self.schedule_tree.heading('Type', text='Type')

        self.schedule_tree.column('Class', width=200)
        self.schedule_tree.column('Faculty', width=150)
        self.schedule_tree.column('Lab', width=100)
        self.schedule_tree.column('Time', width=200)
        self.schedule_tree.column('Type', width=100)

        # Add scrollbars
        schedule_scrollbar_y = ttk.Scrollbar(schedule_display_frame,
orient='vertical',

command=self.schedule_tree.yview)
        schedule_scrollbar_x = ttk.Scrollbar(schedule_display_frame,
orient='horizontal',
```

```
command=self.schedule_tree.xview)


self.schedule_tree.configure(yscrollcommand=schedule_scrollbar_y.set,

xscrollcommand=schedule_scrollbar_x.set)

        self.schedule_tree.pack(side='left', fill='both', expand=True)
        schedule_scrollbar_y.pack(side='right', fill='y')
        schedule_scrollbar_x.pack(side='bottom', fill='x')

    def setup_analysis_tab(self, notebook):
        """Setup of analysis and reporting tab"""
        analysis_frame = ttk.Frame(notebook)
        notebook.add(analysis_frame, text="Analysis & Reports")

        # Analysis buttons
        button_frame = ttk.Frame(analysis_frame)
        button_frame.pack(fill='x', padx=10, pady=10)

        # Generate report button and lab utilization analysis button
        ttk.Button(button_frame, text="Generate Report",
                   command=self.generate_report).pack(side='left',
padx=5)
        ttk.Button(button_frame, text = "Conflicts",

command=self.generate_report).pack(side='left',padx=5)
        ttk.Button(button_frame, text="Lab Utilization",
                   command=self.analyze_utilization).pack(side='left',
padx=5)

        # Report display
        report_frame = ttk.LabelFrame(analysis_frame, text="Analysis
Results")
        report_frame.pack(fill='both', expand=True, padx=10, pady=10)

        self.report_text = scrolledtext.ScrolledText(report_frame,
wrap=tk.WORD)
        self.report_text.pack(fill='both', expand=True, padx=5,
pady=5)

    def generate_schedule(self):
        """Generate the laboratory schedule with constraints"""
        self.status_var.set("Generating schedule...")
        self.progress_var.set(0)
        self.root.update()

        schedule_data = []
        assigned_classes = set()
```

```
        faculty_schedule = {fac_id: {} for fac_id in
self.faculty['id']}
        lab_schedule = {lab: {} for lab in self.labs.keys()}

        max_attempts = int(self.max_attempts_var.get())
        priority = self.priority_var.get()

        # Sort classes based on priority
        classes_to_schedule = self.classes.copy()
        if priority == "concentration_first":
            classes_to_schedule =
classes_to_schedule.sort_values('type',
                key=lambda x: x.map({'concentration': 0, 'double':
1}))
        elif priority == "double_first":
            classes_to_schedule =
classes_to_schedule.sort_values('type',
                key=lambda x: x.map({'double': 0, 'concentration':
1}))
        else:
            classes_to_schedule = classes_to_schedule.sample(frac=1)
# Random order

        total_classes = len(self.classes)

        for idx, (_, class_info) in
enumerate(classes_to_schedule.iterrows()):
            # Update progress
            progress = (idx / total_classes) * 100
            self.progress_var.set(progress)
            self.root.update()

            if class_info['id'] in assigned_classes:
                continue

            scheduled = False
            attempts = 0

            while not scheduled and attempts < max_attempts:
                attempts += 1

                # Select random faculty and time slot
                available_faculty = self.faculty.sample(n=1).iloc[0]
                time_slot = random.choice(self.time_slots)

                # Find suitable lab
                suitable_labs = self.get_suitable_labs(class_info)
                if not suitable_labs:
                    break
```

```
                lab = random.choice(suitable_labs)

                # Check all constraints
                if self.check_scheduling_constraints(class_info,
available_faculty,
                                                     lab, time_slot,
faculty_schedule,
                                                     lab_schedule,
schedule_data):

                    # Schedule a class
                    duration = class_info['duration']
                    for hour_offset in range(duration):
                        current_slot =
self.get_next_time_slot(time_slot, hour_offset)
                        if current_slot:
                            schedule_entry = {
                                'class_id': class_info['id'],
                                'class_name': class_info['name'],
                                'faculty_id': available_faculty['id'],
                                'faculty_name':
available_faculty['name'],
                                'lab': lab,
                                'time_slot': current_slot,
                                'class_type': class_info['type'],
                                'semester': class_info['semester']
                            }
                            # Add to the schedule entry list
                            schedule_data.append(schedule_entry)

                            # Update tracking structures
                            if current_slot not in
faculty_schedule[available_faculty['id']]:

faculty_schedule[available_faculty['id']][current_slot] = []
                            faculty_schedule[available_faculty['id']]
[current_slot].append(class_info['id'])

                            if current_slot not in lab_schedule[lab]:
                                lab_schedule[lab][current_slot] = []
                            lab_schedule[lab]
[current_slot].append(class_info['id'])

                    assigned_classes.add(class_info['id'])
                    scheduled = True

                    # Handle double classes on preferred days
                    if class_info['type'] == 'double' and
```

```
class_info['preferred_days']:
                        self.schedule_double_class_second_session(
                            class_info, available_faculty, lab,
time_slot,
                            schedule_data, faculty_schedule,
lab_schedule
                        )

        # Create schedule DataFrame
        self.schedule = pd.DataFrame(schedule_data)

        # Update display
        self.update_schedule_display()

        success_rate = len(assigned_classes) / total_classes * 100
        self.status_var.set(f"Schedule generated:
{len(assigned_classes)}/{total_classes} classes scheduled
({success_rate:.1f}%)")
        self.progress_var.set(100)

    def get_suitable_labs(self, class_info):
        """Get list of suitable labs for a class"""
        suitable_labs = []

        for lab_name, lab_info in self.labs.items():
            # Check capacity
            if lab_info['capacity'] < class_info['capacity_needed']:
                continue

            # Check type constraints
            if (self.constraint_vars['concentration_small_labs'].get()
and
                class_info['type'] == 'concentration' and
                lab_info['type'] != 'concentration'):
                continue

            suitable_labs.append(lab_name)

        return suitable_labs

    def check_scheduling_constraints(self, class_info, faculty, lab,
time_slot,
                                     faculty_schedule, lab_schedule,
schedule_data):
        """Check if scheduling violates any constraints"""

        # Check faculty availability
        if (self.constraint_vars['faculty_availability'].get() and
            time_slot not in faculty['availability']):
            return False
```

```
        # Check lab conflicts
        if time_slot in lab_schedule[lab] and lab_schedule[lab]
[time_slot]:
            return False

        # Check faculty conflicts
        if time_slot in faculty_schedule[faculty['id']] and
faculty_schedule[faculty['id']][time_slot]:
            return False

        # Check same semester time conflicts
        if self.constraint_vars['no_time_conflict'].get():
            for scheduled in schedule_data:
                if (scheduled['semester'] == class_info['semester']
and
                    scheduled['time_slot'] == time_slot and
                    scheduled['class_id'] != class_info['id']):
                    return False

        return True

    def get_next_time_slot(self, time_slot, hour_offset):
        """Get the time slot offset by specified hours"""
        if hour_offset == 0:
            return time_slot

        # Parse the time slot
        parts = time_slot.split(' ')
        day = parts[0]
        time_range = parts[1]
        start_hour = int(time_range.split(':')[0])

        new_hour = start_hour + hour_offset
        if new_hour >= 19:  # Don't schedule past 7 PM, since the last
class start at 7 and finishes at 9pm
            return None

        return f"{day} {new_hour:02d}:00-{new_hour+2:02d}:00"

    def schedule_double_class_second_session(self, class_info,
faculty, lab, first_time_slot,
                                             schedule_data,
faculty_schedule, lab_schedule):
        """Schedule the second session of a double class"""
        if not class_info['preferred_days']:
            return

        # Find the corresponding day
```

```
        # M/W or T/R classes
        first_day = first_time_slot.split(' ')[0]
        if first_day == 'Monday':
            second_day = 'Wednesday'
        elif first_day == 'Wednesday':
            second_day = 'Monday'
        elif first_day == 'Tuesday':
            second_day = 'Thursday'
        elif first_day == 'Thursday':
            second_day = 'Tuesday'
        else:
            return

        # Get same time on the other day
        time_part = first_time_slot.split(' ')[1]
        second_time_slot = f"{second_day} {time_part}"

        # Check if this slot is available
        if (second_time_slot in lab_schedule[lab] and
lab_schedule[lab][second_time_slot]) or \
           (second_time_slot in faculty_schedule[faculty['id']] and
faculty_schedule[faculty['id']][second_time_slot]):
            return

        # Schedule second session
        schedule_entry = {
            'class_id': class_info['id'] + '_B',
            'class_name': class_info['name'] + ' (Session B)',
            'faculty_id': faculty['id'],
            'faculty_name': faculty['name'],
            'lab': lab,
            'time_slot': second_time_slot,
            'class_type': class_info['type'],
            'semester': class_info['semester']
        }

        schedule_data.append(schedule_entry)

        # Update tracking
        if second_time_slot not in faculty_schedule[faculty['id']]:
            faculty_schedule[faculty['id']][second_time_slot] = []
        faculty_schedule[faculty['id']]
[second_time_slot].append(class_info['id'] + '_B')

        if second_time_slot not in lab_schedule[lab]:
            lab_schedule[lab][second_time_slot] = []
        lab_schedule[lab][second_time_slot].append(class_info['id'] +
'_B')
```

```
    def update_schedule_display(self):
        """Update the schedule display in the GUI"""
        # Clear existing items
        for item in self.schedule_tree.get_children():
            self.schedule_tree.delete(item)

        # Add schedule items
        if not self.schedule.empty:
            for _, row in self.schedule.iterrows():
                self.schedule_tree.insert('', 'end', values=(
                    row['class_name'], row['faculty_name'],
row['lab'],
                    row['time_slot'], row['class_type']
                ))

    def clear_schedule(self):
        """Clear the current schedule"""
        self.schedule = pd.DataFrame()
        self.update_schedule_display()
        self.status_var.set("Schedule cleared")
        self.progress_var.set(0)

    def generate_report(self):
        """Generate a comprehensive report"""
        if self.schedule.empty:
            messagebox.showwarning("No Schedule", "Please generate a
schedule first.")
            return

        report = []
        report.append("UISU-A LKB Building Laboratory Scheduling
Report")
        report.append("=" * 50)
        report.append(f"Generated on: {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}")
        report.append(f"Total scheduled classes:
{len(self.schedule)}")
        report.append("")

        # Schedule summary by type
        type_counts = self.schedule['class_type'].value_counts()
        report.append("Classes by Type:")
        for class_type, count in type_counts.items():
            report.append(f"  {class_type.title()}: {count}")
        report.append("")

        # Lab utilization
        lab_counts = self.schedule['lab'].value_counts()
        report.append("Lab Utilization:")
```

```
        for lab, count in lab_counts.items():
            utilization = (count / len(self.time_slots)) * 100
            report.append(f"  {lab}: {count} classes
({utilization:.1f}% utilization)")
        report.append("")

        # Faculty workload
        faculty_counts = self.schedule['faculty_name'].value_counts()
        report.append("Faculty Workload (Top 10):")
        for faculty, count in faculty_counts.head(10).items():
            report.append(f"  {faculty}: {count} classes")
        report.append("")

        # Time slot distribution
        time_counts = self.schedule['time_slot'].value_counts()
        report.append("Popular Time Slots (Top 10):")
        for time_slot, count in time_counts.head(10).items():
            report.append(f"  {time_slot}: {count} classes")

        self.report_text.delete(1.0, tk.END)
        self.report_text.insert(tk.END, "\n".join(report))

    def check_conflicts(self):
        """Check for scheduling conflicts"""
        if self.schedule.empty:
            messagebox.showwarning("No Schedule", "Please generate a
schedule first.") # Pop up to let the user know that no schedule has
been generated
            return

        conflicts = []

        # Check for time conflicts
        for time_slot in self.schedule['time_slot'].unique():
            slot_classes = self.schedule[self.schedule['time_slot'] ==
time_slot]

            # Same lab conflicts
            lab_conflicts = slot_classes.groupby('lab').size()
            for lab, count in lab_conflicts.items():
                if count > 1:
                    conflicts.append(f"Lab conflict in {lab} at
{time_slot}: {count} classes")

            # Same faculty conflicts
            faculty_conflicts =
slot_classes.groupby('faculty_name').size()
            for faculty, count in faculty_conflicts.items():
                if count > 1:
```

```
                    conflicts.append(f"Faculty conflict for {faculty}
at {time_slot}: {count} classes")

        # Check capacity violations
        for _, row in self.schedule.iterrows():
            class_info = self.classes[self.classes['id'] ==
row['class_id']].iloc[0]
            lab_capacity = self.labs[row['lab']]['capacity']
            if class_info['capacity_needed'] > lab_capacity:
                conflicts.append(f"Capacity violation:
{row['class_name']} needs {class_info['capacity_needed']} but
{row['lab']} has {lab_capacity}")

        result = "Conflict Analysis Results:\n\n"
        if conflicts:
            result += f"Found {len(conflicts)} conflicts:\n\n"
            result += "\n".join(conflicts)
        else:
            result += "No conflicts found! Schedule appears valid."

        self.report_text.delete(1.0, tk.END)
        self.report_text.insert(tk.END, result)

    def analyze_utilization(self):
        """Analyze laboratory utilization"""
        if self.schedule.empty:
            messagebox.showwarning("No Schedule", "Please generate a
schedule first.")
            return

        analysis = []
        analysis.append("Laboratory Utilization Analysis")
        analysis.append("=" * 40)
        analysis.append("")

        total_slots = len(self.time_slots)

        for lab_name, lab_info in self.labs.items():
            lab_schedule = self.schedule[self.schedule['lab'] ==
lab_name]
            used_slots = len(lab_schedule)
            utilization = (used_slots / total_slots) * 100

            analysis.append(f"{lab_name}:")
            analysis.append(f"  Capacity: {lab_info['capacity']}
students")
            analysis.append(f"  Type: {lab_info['type']}")
            analysis.append(f"  Used slots:
{used_slots}/{total_slots}")
```

```
            analysis.append(f"  Utilization: {utilization:.1f}%")

            if used_slots > 0:
                avg_class_size = lab_schedule.apply(lambda row:
                    self.classes[self.classes['id'] ==
row['class_id']]['capacity_needed'].iloc[0]
                    if not self.classes[self.classes['id'] ==
row['class_id']].empty else 0, axis=1).mean()
                analysis.append(f"  Average class size:
{avg_class_size:.1f} students")

            analysis.append("")

        # Overall statistics
        total_scheduled = len(self.schedule)
        total_possible = len(self.labs) * total_slots
        overall_utilization = (total_scheduled / total_possible) * 100

        analysis.append("Overall Statistics:")
        analysis.append(f"  Total scheduled classes:
{total_scheduled}")
        analysis.append(f"  Total possible slots: {total_possible}")
        analysis.append(f"  Overall utilization:
{overall_utilization:.1f}%")

        self.report_text.delete(1.0, tk.END)
        self.report_text.insert(tk.END, "\n".join(analysis))

    def regenerate_data(self):
        """Regenerate sample data"""
        # Sample classes and faculty is generated
        self.classes = self.generate_sample_classes()
        self.faculty = self.generate_sample_faculty()
        self.clear_schedule()
        messagebox.showinfo("Data Regenerated", "New sample data has
been generated.")

    def export_data(self):
        """Export data to files"""
        try:
            self.classes.to_csv('classes_data.csv', index=False)
            self.faculty.to_csv('faculty_data.csv', index=False)

            with open('labs_data.json', 'w') as f:
                json.dump(self.labs, f, indent=2)

            messagebox.showinfo("Export Complete", "Data exported to
CSV and JSON files.")
        except Exception as e:
```

```
            messagebox.showerror("Export Error", f"Error exporting
data: {str(e)}")

    def export_schedule(self):
        """Export schedule to CSV"""
        if self.schedule.empty:
            messagebox.showwarning("No Schedule", "Please generate a
schedule first.")
            return

        try:
            filename = f'lab_schedule_{datetime.now().strftime("%Y%m
%d_%H%M%S")}.csv'
            self.schedule.to_csv(filename, index=False)
            messagebox.showinfo("Export Complete", f"Schedule exported
to {filename}")
        except Exception as e:
            messagebox.showerror("Export Error", f"Error exporting
schedule: {str(e)}")


def main():
    """Main function of the application"""
    root = tk.Tk()
    app = LaboratoryScheduler(root)

    # Add menu bar
    menubar = tk.Menu(root)
    root.config(menu=menubar)

    # File menu
    file_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="File", menu=file_menu)
    file_menu.add_command(label="Generate New Data",
command=app.regenerate_data)
    file_menu.add_command(label="Export Data",
command=app.export_data)
    file_menu.add_separator()
    file_menu.add_command(label="Exit", command=root.quit)

    # Schedule menu
    schedule_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Schedule", menu=schedule_menu)
    schedule_menu.add_command(label="Generate Schedule",
command=app.generate_schedule)
    schedule_menu.add_command(label="Clear Schedule",
command=app.clear_schedule)
    schedule_menu.add_command(label="Export Schedule",
command=app.export_schedule)
```

```
    # Analysis menu
    analysis_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Analysis", menu=analysis_menu)
    analysis_menu.add_command(label="Generate Report",
command=app.generate_report)
    analysis_menu.add_command(label="Check Conflicts",
command=app.check_conflicts)
    analysis_menu.add_command(label="Lab Utilization",
command=app.analyze_utilization)

    # Help menu
    help_menu = tk.Menu(menubar, tearoff=0)
    menubar.add_cascade(label="Help", menu=help_menu)
    help_menu.add_command(label="About", command=lambda:
messagebox.showinfo(
        "About", "UISU-A LKB Building Laboratory Scheduler\n\nA
comprehensive scheduling system for laboratory classes with constraint
handling."))

    root.mainloop()


if __name__ == "__main__":
    main()
```